Regression with statsmodels

Formula API vs Matrix API

IEOR E4576 — Data-Driven Models in Finance — Fall 2026 — Week 4

Contents

1 Installation and Import 1

2 Loading Data 1

3 Formula API 1
- 3.1 Simple Regression . . . 2
- 3.2 Multiple Regression . . . 2

4 Matrix API 3
- 4.1 Simple Regression . . . 3
- 4.2 Multiple Regression . . . 3

5 Comparison Summary 3

6 Accessing Results 4

1 Installation and Import

Install the packages from the shell:

```
pip install statsmodels pandas
```

Then import them in Python:

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
import pandas as pd
import numpy as np
```

2 Loading Data

We use the `mtcars` dataset throughout to demonstrate both API styles.

```
# Load mtcars dataset
mtcars = sm.datasets.get_rdataset("mtcars", "datasets").data
print(mtcars[['mpg', 'wt', 'hp']].head())
```

```
                   mpg    wt     hp
Mazda RX4         21.0  2.620  110
Mazda RX4 Wag     21.0  2.875  110
Datsun 710        22.8  2.320   93
Hornet 4 Drive    21.4  3.215  110
Hornet Sportabout 18.7  3.440  175
```

3 Formula API

The Formula API uses R-style formula strings. The intercept is added automatically.

3.1 Simple Regression

```python
from statsmodels.formula.api import ols

# Simple regression: mpg vs weight
model = ols('mpg ~ wt', data=mtcars).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.753
Model:                            OLS   Adj. R-squared:                  0.745
Method:                 Least Squares   F-statistic:                     91.38
No. Observations:                  32   Prob (F-statistic):           1.29e-10
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     37.2851      1.878     19.858      0.000      33.450      41.120
wt            -5.3445      0.559     -9.559      0.000      -6.486      -4.203
==============================================================================
```

3.2 Multiple Regression

```python
# Multiple regression: mpg vs weight and horsepower
model = ols('mpg ~ wt + hp', data=mtcars).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.827
Model:                            OLS   Adj. R-squared:                  0.815
Method:                 Least Squares   F-statistic:                     69.21
No. Observations:                  32   Prob (F-statistic):           9.11e-12
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     37.2273      1.599     23.285      0.000      33.957      40.497
wt            -3.8778      0.633     -6.129      0.000      -5.172      -2.584
hp            -0.0318      0.009     -3.519      0.001      -0.050      -0.013
==============================================================================
```

4 Matrix API

The Matrix API takes numpy arrays or pandas DataFrames directly.

Add the intercept yourself. Unlike the Formula API, `sm.OLS` does not include an intercept. You must add a column of ones with `sm.add_constant(X)`; otherwise the regression is forced through the origin.

4.1 Simple Regression

```python
import statsmodels.api as sm

# Prepare data - MUST add constant for intercept
X = mtcars[['wt']]
X = sm.add_constant(X)  # Adds column of 1s for intercept
y = mtcars['mpg']

# Fit model
model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.753
Model:                            OLS   Adj. R-squared:                  0.745
Method:                 Least Squares   F-statistic:                     91.38
No. Observations:                  32   Prob (F-statistic):           1.29e-10
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         37.2851      1.878     19.858      0.000      33.450      41.120
wt            -5.3445      0.559     -9.559      0.000      -6.486      -4.203
==============================================================================
```

4.2 Multiple Regression

```python
# Multiple predictors
X = mtcars[['wt', 'hp']]
X = sm.add_constant(X)  # Adds column of 1s for intercept
y = mtcars['mpg']

model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.827
Model:                            OLS   Adj. R-squared:                  0.815
Method:                 Least Squares   F-statistic:                     69.21
No. Observations:                  32   Prob (F-statistic):           9.11e-12
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         37.2273      1.599     23.285      0.000      33.957      40.497
wt            -3.8778      0.633     -6.129      0.000      -5.172      -2.584
hp            -0.0318      0.009     -3.519      0.001      -0.050      -0.013
==============================================================================
```

The estimates are identical to the Formula API; only the intercept label differs (`Intercept` vs `const`).

5 Comparison Summary

Feature	Formula API	Matrix API
Import	`from statsmodels.formula.api import ols`	`import statsmodels.api as sm`
Intercept	Automatic	Manual: `sm.add_constant(X)`
Syntax	`ols('y ~ x1 + x2', data)`	`sm.OLS(y, X)`
Input	DataFrame + formula string	Arrays or DataFrames
Best for	Quick exploration	Production code

6 Accessing Results

Both APIs return the same results object:

```python
print(f"Coefficients:\n{model.params}")
print(f"R-squared: {model.rsquared:.4f}")
print(f"F-statistic: {model.fvalue:.2f} (p={model.f_pvalue:.2e})")
print(f"Confidence Intervals:\n{model.conf_int()}")

# Predictions
predictions = model.predict(X_new)  # X_new must include constant for Matrix API
residuals = model.resid
```